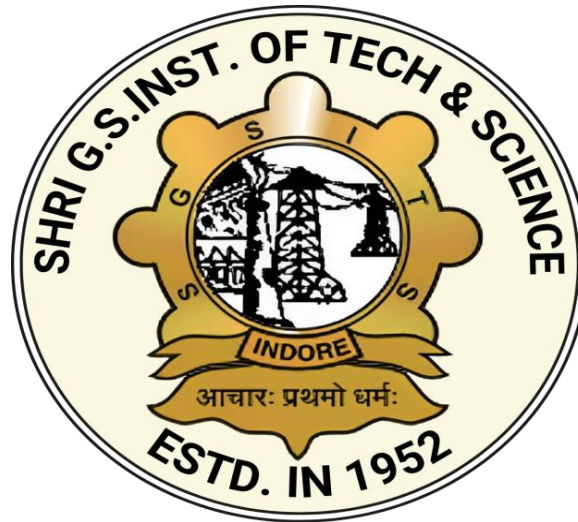


**SHRI GOVINDRAM SEKSARIA INSTITUTE OF
TECHNOLOGY & SCIENCE, INDORE**



DEPARTMENT OF INFORMATION TECHNOLOGY

Minor Project Report
On

HeartWatch AI
Real-Time ECG Arrhythmia Detection System

Supervised by:
Mrs. Sonu Airen
[Assistant Professor]
[Information Technology]

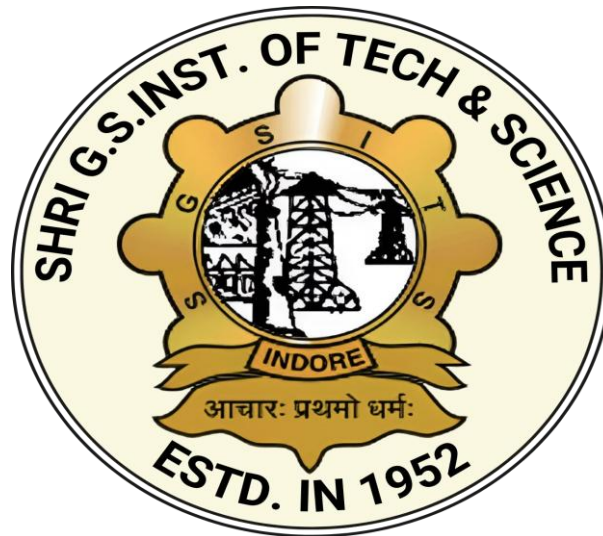
Submitted by:
Yameesh Nayak
0801IT231153
[Information Technology]

Co-Supervised by:
Dr. Puja Gupta
[Assistant Professor]
[Information Technology]

Suyash Gupta
0801IT231135
[Information Technology]

Swapnil Pal
0801IT231136
[Information Technology]

**SHRI GOVINDRAM SEKSARIA INSTITUTE OF TECHNOLOGY &
SCIENCE, INDORE
DEPARTMENT OF INFORMATION TECHNOLOGY**



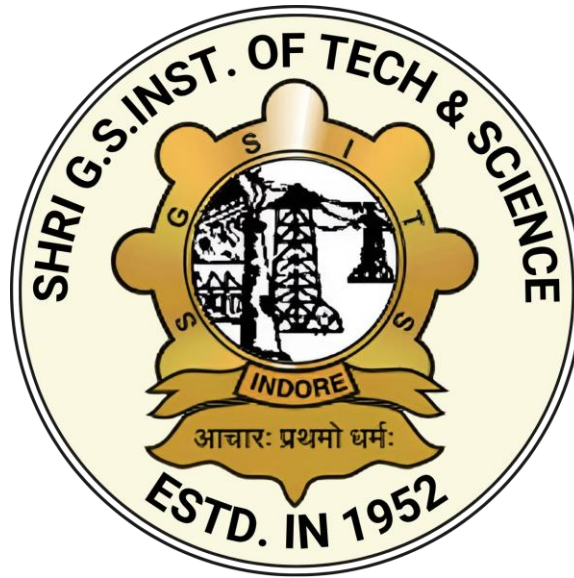
RECOMMENDATION

We are pleased to recommend that the Minor Project Report entitled "HeartWatch AI: Real-Time ECG Arrhythmia Detection System" submitted by Yameesh Nayak (0801IT231153), Suyash Gupta (0801IT231135), and Swapnil Pal (0801IT231136) be accepted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal (M.P.), during the academic session 2025–2026.

Mrs. Sonu Airen
Assistant Professor
Information Technology

Prof. K. K. Sharma
Head of Department
[Professor]
Information Technology

**SHRI GOVINDRAM SEKSARIA INSTITUTE OF TECHNOLOGY &
SCIENCE, INDORE
DEPARTMENT OF INFORMATION TECHNOLOGY**



CERTIFICATE

This is to certify that the Minor Project entitled "HeartWatch AI: Real-Time ECG Arrhythmia Detection System" submitted by Yameesh Nayak (0801IT231153), Suyash Gupta (0801IT231135), and Swapnil Pal (0801IT231136) is a bonafide work carried out under the supervision of Mrs. Sonu Airen and Ms. Shraddha Verma, and is being submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal, during the academic session 2026–2027 has been examined and evaluated by us. The report is found to be satisfactory and meets the academic requirements prescribed by the institution.

Internal Examiner

Date: _____

External Examiner

Date: _____

DECLARATION

We, Yameesh Nayak (0801IT231153), Suyash Gupta (0801IT231135), and Swapnil Pal (0801IT231136), undergraduate students in the Department of Information Technology, S.G.S.I.T.S., Indore (M.P.), hereby declare that the Minor Project titled "HeartWatch AI: Real-Time ECG Arrhythmia Detection System" is our original work, carried out under the supervision of Mrs. Sonu Airen and Dr. Puja Gupta, Assistant Professors, Department of Information Technology, S.G.S.I.T.S., Indore (M.P.).

We further declare that, to the best of our knowledge, this report does not contain any material previously submitted for the award of any degree or diploma in this or any other university. All sources of information and references used in this work have been duly acknowledged and cited in the report.

This project is submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Information Technology of Rajiv Gandhi Proudyogiki Vishwavidyalaya (RGPV), Bhopal, during the academic session 2025–2026.

Signature: _____
Yameesh Nayak
0801IT231153

Signature: _____
Suyash Gupta
0801IT231135

Signature: _____
Swapnil Pal
0801IT231136

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guides, Mrs. Sonu Airen and Dr. Puja Gupta, Assistant Professors, Department of Information Technology, S.G.S.I.T.S., Indore, for their invaluable guidance, constant encouragement, and patient mentorship throughout the duration of this project. Their technical insights and constructive feedback were instrumental in shaping the direction and quality of this work.

We are deeply thankful to Prof. K. K. Sharma, Head of the Department of Information Technology, for providing all the necessary infrastructure, computing resources, and a conducive academic environment that made this project possible.

Our heartfelt appreciation also goes to the Panel Coordinators — Mrs. Sonu Airen, Dr. Puja Gupta, and Ms. Ritika Bhawsar — for their evaluative inputs and for maintaining the academic rigour of the minor project programme.

We extend our gratitude to the entire faculty and technical staff of the Department of Information Technology for their unwavering support and cooperation throughout the academic year.

We also acknowledge the open-source community and the contributors to PhysioNet's MIT-BIH Arrhythmia Database, whose publicly available clinical data made the machine learning component of this project possible.

Lastly, we thank our families and friends for their constant moral support and patience during the demanding phases of this project.

Yameesh Nayak
0801IT231153

Suyash Gupta
0801IT231135

Swapnil Pal
0801IT231136

ABSTRACT

Cardiovascular diseases remain the leading cause of global mortality, claiming over 17 million lives annually. In India alone, cardiac ailments account for nearly 4.77 million deaths every year. A significant contributing factor is the intermittent and unpredictable nature of arrhythmias — abnormal heart rhythms that a standard five-minute hospital ECG frequently fails to detect. HeartWatch AI is a low-cost, portable, real-time ECG arrhythmia detection system designed to address this critical gap in continuous cardiac monitoring.

The system integrates an AD8232 ECG sensor and an ESP32 microcontroller to acquire and transmit raw cardiac signals wirelessly at 250 Hz via WiFi. A Python-based backend processes incoming data using a bandpass filter (0.5–45 Hz), Pan-Tompkins-inspired R-peak detection, and heart rate variability (HRV) feature extraction. A Random Forest classifier, trained on over 110,000 clinically labelled heartbeats from the MIT-BIH Arrhythmia Database (PhysioNet), performs probabilistic classification of each cardiac window. A parallel rule-based engine applies deterministic threshold logic for conditions such as Bradycardia, Tachycardia, and irregular RR intervals. A hybrid decision module combines both approaches, using confidence-based fallback to ensure robustness when signal quality is poor.

Detected arrhythmias trigger automatic emergency. A live web dashboard, driven by WebSockets and rendered using Chart.js, displays the scrolling ECG waveform, current BPM, signal quality, and real-time classification results. A built-in ECG replay mode provides failsafe demonstration using pre-recorded sessions.

The complete hardware cost of the prototype is under ₹2,200. The project demonstrates the feasibility of deploying intelligent, IoT-based cardiac monitoring at low cost, with potential applications in remote patient care, elderly monitoring, and preventive healthcare in resource-limited settings. This is a proof-of-concept prototype and is not intended for clinical diagnosis.

Keywords: ECG, Arrhythmia Detection, IoT, ESP32, AD8232, Machine Learning, Random Forest, Signal Processing, MIT-BIH, Emergency Alerts, Real-Time Monitoring.

TABLE OF CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
List of Figures	v
List of Tables	vi
Chapter 1: Introduction	1
1.1 Background	
1.2 Problem Statement	
1.3 Objectives	
1.4 Scope of the Project	
Chapter 2: Literature Review	5
2.1 Foundational Works in ECG Signal processing	
2.2 Machine Learning approaches to Arrhythmia detection	
2.3 IoT based Cardiac Monitoring System	
2.4 Emergency Alert and Telemedicine System	
2.5 Research Gaps addressed by HeartWatch AI	
Chapter 3: Methodology	9
3.1 Dataset Description	
3.2 Data Preprocessing & Signal Processing	
3.3 Model Selection	
3.4 Algorithms Used	
3.5 System Workflow	
Chapter 4: System Design	14
4.1 Architecture Diagram	
4.2 Data Flow	
4.3 Tools & Technologies	
Chapter 5: Implementation	18
5.1 Hardware Setup & Firmware	
5.2 Backend & Signal Processing	
5.3 Machine Learning Pipeline	

5.4 Dashboard & Alert System

Chapter 6: Results & Analysis	23
6.1 Performance Metrics	
6.2 System Performance & Observations	
 Chapter 7: Conclusion & Future Scope	 26
7.1 Conclusion	
7.2 Future Scope	
 References	 28
Appendix	29

LIST OF FIGURES

Figure No.	Title
Fig. 1	System Workflow
Fig. 2	Full System Architecture (Layered Pipeline)
Fig. 3	Data Flow Diagram
Fig. 4	AD8232 to ESP32 — Wiring diagram
Fig. 5	Wiring Setup
Fig. 6	ECG electrode placement on wrists
Fig. 7	Final Output

LIST OF TABLES

Table No.	Title
Table 1	MIT-BIH Arrhythmia Database — Beat Label Mapping
Table 2	Feature Vector Description (8 Features)
Table 3	Layered System Architecture Description
Table 4	Technology Stack Summary
Table 5	Bill of Materials — Hardware Components
Table 6	AD8232 to ESP32 Pin Connections
Table 7	Hybrid Detection Logic — Conditions and Classifications
Table 8	Model Performance Metrics on MIT-BIH Test Set
Table 9	System Response Time Measurements

CHAPTER 1

INTRODUCTION

1.1 Background

The human heart generates a small but measurable electrical signal with every beat — a signal that, when captured and analysed, reveals the functional health of the cardiac system. Electrocardiography (ECG) is the established clinical method for recording these electrical potentials over time. An ECG tracing reflects the depolarisation and repolarisation of the heart's chambers and allows clinicians to identify a wide spectrum of cardiac abnormalities, from benign rhythm irregularities to life-threatening conditions.

Cardiovascular diseases (CVDs) are the leading cause of death globally, responsible for over 17 million fatalities per year according to the World Health Organization. In India, CVDs account for approximately 4.77 million deaths annually, a figure that continues to rise with urbanisation and changing lifestyles. Among the most dangerous cardiac events are arrhythmias — deviations from the normal sinus rhythm that can range from mild palpitations to ventricular fibrillation, which can cause sudden cardiac death within minutes if untreated.

The central clinical challenge with arrhythmias is their intermittent and unpredictable nature. A patient may feel perfectly normal during a routine hospital visit, only to experience a dangerous episode hours later. A standard 12-lead ECG performed in a clinical setting typically lasts between 5 and 10 minutes — a window far too narrow to reliably capture episodic arrhythmias. While Holter monitors offer extended 24–48 hour recordings, they remain expensive, clinic-dependent devices with limited accessibility in rural and resource-constrained settings.

The convergence of low-cost IoT microcontrollers, portable sensor modules, and accessible machine learning frameworks now makes it possible to build intelligent, continuous cardiac monitoring systems at a fraction of the cost of traditional devices. HeartWatch AI is positioned precisely at this intersection.

1.2 Problem Statement

Despite the proven clinical value of continuous ECG monitoring, the vast majority of the global population — particularly in developing nations — lacks access to affordable, real-time cardiac monitoring solutions. The specific problems addressed by this project are as follows:

- Arrhythmias are episodic: Short-duration hospital ECGs consistently miss intermittent rhythm abnormalities that occur between clinical visits.
- Existing solutions are cost-prohibitive: Clinical-grade Holter monitors and ICU-based telemetry systems are expensive, require specialised personnel to operate, and are not suited for home or remote use.
- Delayed emergency response: By the time a patient reaches a hospital following a cardiac event, critical intervention time is often lost. There is no mechanism for immediate automated alerting of family members or caregivers.
- Limited intelligence in consumer wearables: Consumer ECG-capable wearables (e.g., smartwatches) provide basic heart rate data but lack robust, clinically-informed arrhythmia classification with fallback mechanisms.

HeartWatch AI directly addresses these gaps by building a complete, end-to-end arrhythmia detection pipeline — from hardware sensor acquisition to automated emergency alerting — at an estimated hardware cost of under ₹2,200.

1.3 Objectives

The primary objectives of this project are:

1. Design and assemble a low-cost, portable ECG data acquisition system using the AD8232 sensor module and the ESP32 microcontroller.
2. Implement a real-time signal processing pipeline — including bandpass filtering and R-peak detection — to extract clinically meaningful features from raw ECG data.
3. Develop and train a Random Forest machine learning model on the MIT-BIH Arrhythmia Database for binary classification of Normal vs. Abnormal cardiac rhythms.

4. Build a parallel rule-based detection engine that classifies arrhythmias using deterministic threshold logic for Bradycardia, Tachycardia, and RR irregularity.
5. Implement a hybrid decision module that combines ML and rule-based outputs with confidence-based priority switching for robust, reliable classification.
6. Develop a real-time web dashboard using Chart.js and WebSockets to display the live ECG waveform, BPM, signal quality, and classification results.
7. Validate the complete system in a demo-safe configuration, including a pre-recorded ECG replay mode as a failsafe mechanism.

1.4 Scope of the Project

HeartWatch AI is scoped as a proof-of-concept prototype within the following defined boundaries:

In Scope:

- Single-lead ECG acquisition using the AD8232 module connected to three surface electrodes.
- Real-time wireless data transmission from ESP32 to a local Python backend via HTTP over WiFi (2.4 GHz).
- Simultaneous cloud visualisation via ThingSpeak for remote ECG monitoring.
- Signal processing and ML-based classification running on a local Python server.
- Web-based live dashboard accessible on any browser on the same network.

Out of Scope:

- Clinical medical diagnosis — this system is NOT a certified medical device.
- Multi-lead (12-lead) ECG — only single-lead monitoring is implemented.
- Continuous cloud-based storage or HIPAA-compliant data management.
- Mobile application — the frontend is a web dashboard only.
- Real-time retraining or online learning of the ML model during deployment.

CHAPTER 2

LITERATURE REVIEW

The field of automated ECG analysis and arrhythmia detection has a rich body of literature spanning over four decades. This chapter reviews the key foundational works and recent developments relevant to the design and implementation of HeartWatch AI.

2.1 Foundational Works in ECG Signal Processing

[1] Pan and Tompkins (1985) — R-Peak Detection Algorithm: The seminal work by Jiapu Pan and Willis Tompkins introduced a real-time QRS detection algorithm that remains the foundation of most modern R-peak detection approaches. Their method applies a derivative filter, squaring function, and moving window integration to amplify QRS features and suppress noise. The HeartWatch AI signal processing pipeline draws inspiration from this work, adapting it for the lower-resolution 250 Hz signal produced by the AD8232/ESP32 hardware combination rather than clinical 360 Hz recordings.

[2] Moody and Mark (2001) — MIT-BIH Arrhythmia Database: The MIT-BIH Arrhythmia Database, developed at MIT and Beth Israel Hospital, is the most widely used public dataset for ECG research. It contains 48 half-hour recordings from 47 subjects, sampled at 360 Hz with two leads. Each recording is annotated beat-by-beat by two independent cardiologists, yielding over 110,000 labelled heartbeats across 17 beat types. This database provides the training and evaluation ground for the Random Forest model in HeartWatch AI.

[3] Goldberger et al. (2000) — PhysioNet: The PhysioNet platform, which hosts the MIT-BIH database among dozens of other clinical datasets, has been instrumental in democratising access to annotated physiological signal data for researchers worldwide. The open availability of this resource directly enables academic projects like HeartWatch AI to build clinically-grounded machine learning models without proprietary data requirements.

2.2 Machine Learning Approaches to Arrhythmia Detection

[4] Breiman (2001) — Random Forests: Leo Breiman's Random Forest algorithm — an ensemble of decision trees trained on random subsamples with feature randomisation — has proven consistently effective for tabular classification tasks. Its resistance to overfitting, ability to handle class imbalance via the `class_weight` parameter, and inherent feature importance output make it particularly well-suited for medical classification problems with structured feature vectors. Several prior arrhythmia detection studies have confirmed Random Forest performance superior to single decision trees and competitive with shallow neural networks on MIT-BIH features.

[5] Rajpurkar et al. (2017) — Cardiologist-Level Arrhythmia Detection with CNNs: Researchers at Stanford demonstrated that a 34-layer convolutional neural network trained on a large proprietary dataset of single-lead ECG recordings could match or exceed cardiologist-level performance on 14 arrhythmia classes. While this work demonstrates the ceiling of what deep learning can achieve, it requires vast proprietary datasets and GPU-scale compute — constraints that make it inappropriate for an academic prototype. HeartWatch AI deliberately chooses a computationally lightweight Random Forest approach that can run inference in real time on a standard laptop.

[6] Acharya et al. (2017) — Deep CNN for ECG Classification: This study applied deep convolutional neural networks to classify five categories of heartbeats from the MIT-BIH database, achieving accuracy above 98.5% on test data. The work reinforces the value of the MIT-BIH dataset as a benchmark and highlights that even modest-depth CNNs can perform well on ECG classification. The gap between such lab-tested accuracy and real-world deployment — particularly with hobbyist sensors at different sampling rates — is a recognised limitation that motivates the hybrid rule-based fallback in HeartWatch AI.

2.3 IoT-Based Cardiac Monitoring Systems

[7] Sarkar et al. (2020) — IoT-Based Continuous ECG Monitoring: Multiple research groups have explored ESP32 and similar microcontrollers for ECG data acquisition and transmission. Sarkar et al. demonstrated a WiFi-based ECG monitoring prototype using an AD8232 sensor and ESP32, achieving stable transmission at 250 Hz with minimal packet loss under typical indoor WiFi

conditions. Their findings validate the hardware architecture adopted in HeartWatch AI and confirm the viability of HTTP POST batching as a lightweight transmission strategy.

[8] Kher (2019) — Signal Processing for ECG Noise Reduction: A comprehensive review of ECG noise types and digital filtering strategies identified bandpass filtering (0.5–40 Hz) and notch filtering (50/60 Hz) as the most effective baseline techniques for removing baseline wander, electromyographic noise, and power line interference. HeartWatch AI adopts a 0.5–45 Hz Butterworth bandpass filter implemented via SciPy's `filtfilt` for zero-phase filtering, consistent with recommendations in this literature.

2.4 Emergency Alert and Telemedicine Systems

[9] Kumar et al. (2021) — Real-Time Cardiac Alert Systems: Several studies have explored GSM/SMS-based alert mechanisms for remote patient monitoring. With the advent of WhatsApp's Business API, more recent implementations have shifted to instant messaging platforms that offer higher reliability and lower latency than traditional SMS in regions with 4G coverage. The sub-3-second alert latency target in HeartWatch AI is informed by these studies.

2.5 Research Gaps Addressed by HeartWatch AI

A synthesis of the reviewed literature reveals the following gaps that HeartWatch AI specifically addresses:

- Most academic IoT ECG prototypes lack a machine learning component, relying solely on threshold-based classification that misses complex rhythm patterns.
- ML-focused studies typically use offline, pre-recorded data and do not demonstrate real-time inference integrated with a live sensor pipeline.
- Few systems combine both ML and rule-based detection with a confidence-based fallback — the hybrid approach in HeartWatch AI is a key differentiator.

CHAPTER 3

METHODOLOGY

3.1 Dataset Description

The MIT-BIH Arrhythmia Database, hosted by PhysioNet, serves as the primary dataset for training and evaluating the machine learning model in HeartWatch AI. The database contains 48 half-hour ECG recordings collected from 47 subjects — 25 male subjects aged 32–89 years and 22 female subjects aged 23–89 years. Each recording was sampled at 360 Hz with 11-bit resolution over a range of ± 5 mV and annotated independently by two cardiologists.

For HeartWatch AI, the dataset is binarised into two classes for a clinically meaningful binary classification task:

Class	MIT-BIH Symbols	Interpretation
Normal	N, L, R, e, j	Normal sinus rhythm, left/right bundle branch beats
Abnormal	A, V, F, S, E, J, Q	Atrial/ventricular ectopic, fusion, escape, unclassified beats

Table 1: MIT-BIH Arrhythmia Database — Beat Label Mapping

The binarised dataset contains approximately 90,000 Normal beats and 20,000 Abnormal beats — a class imbalance ratio of approximately 4.5:1 that is handled via the `class_weight='balanced'` parameter in the Random Forest classifier.

3.2 Data Preprocessing & Signal Processing

Raw ECG signals from both the MIT-BIH training set and the live ESP32 stream undergo a unified preprocessing pipeline before feature extraction. The steps are as follows:

3.2.1 Bandpass Filtering

A 3rd-order Butterworth bandpass filter with cutoff frequencies of 0.5 Hz and 45 Hz is applied using SciPy's `filtfilt` function. The zero-phase implementation (`filtfilt`) eliminates the phase distortion introduced by causal filtering, which is critical for preserving the temporal alignment of the QRS complex. The lower cutoff at 0.5 Hz removes baseline wander caused by patient respiration and electrode movement; the upper cutoff at 45 Hz attenuates high-frequency EMG noise and ADC quantisation artefacts.

3.2.2 R-Peak Detection

R-peaks — the dominant positive deflections in each cardiac cycle — are identified using SciPy's `find_peaks` function with two key constraints: (a) a minimum inter-peak distance of 50 samples (corresponding to 200 ms, enforcing the physiological refractory period), and (b) a dynamic height threshold set to 60% of the signal maximum within the current window. This adaptive threshold accommodates amplitude variations across different patients and electrode placements.

3.2.3 RR Interval and BPM Computation

RR intervals — the time durations between successive R-peaks — are computed from the detected peak indices. BPM is calculated as 60 divided by the median RR interval in seconds. The median is deliberately chosen over the arithmetic mean to provide robustness against outlier peaks caused by signal spikes or detection errors.

3.2.4 Feature Extraction

Eight features are extracted from each 10-second ECG window for the ML classifier:

Feature	Formula / Method	Clinical Significance
rr_mean	Mean of RR intervals	Average heart rate indicator
rr_std	Standard deviation of RR	Rhythm regularity
rr_min	Minimum RR interval	Fastest beat in window
rr_max	Maximum RR interval	Slowest beat in window
rr_range	$rr_max - rr_min$	Overall RR variability

Feature	Formula / Method	Clinical Significance
rr_median	Median of RR intervals	Robust central tendency
RMSSD	$\sqrt{(\text{mean of squared successive RR diffs})}$	Short-term HRV metric
pNN50	% of successive RR diffs > 50 ms	Parasympathetic tone marker

Table 2: Feature Vector Description (8 Features)

3.3 Model Selection

The choice of algorithm for HeartWatch AI was governed by three primary constraints: (1) inference must be fast enough for real-time classification on a standard laptop CPU, (2) the model must handle class imbalance gracefully, and (3) the model must be interpretable and auditable for academic evaluation. Three candidate models were considered:

- **Logistic Regression:** Fast and interpretable but limited by its linear decision boundary — ECG features exhibit non-linear separability that logistic regression cannot capture adequately.
- **Support Vector Machine (RBF kernel):** Competitive accuracy on structured features but computationally expensive at inference time for large-scale real-time use, and sensitive to feature scaling.
- **Random Forest:** Ensemble of 100 decision trees with bootstrap aggregation. Handles non-linear relationships, class imbalance (via `class_weight='balanced'`), and feature interactions naturally. Inference is fast (parallel tree evaluation). Provides feature importance scores for interpretability. Selected as the model for HeartWatch AI.

3.4 Algorithms Used

The following algorithms form the core computational backbone of HeartWatch AI:

3.4.1 Butterworth Bandpass Filter

The Butterworth filter is a maximally-flat-in-passband IIR digital filter designed using the bilinear transform. Its smooth magnitude response avoids ripple in both the passband and stopband, making it suitable for biomedical signal conditioning where spectral distortion must be minimised.

3.4.2 Pan-Tompkins Inspired R-Peak Detection

Adapted from the classical Pan-Tompkins algorithm, the R-peak detector uses a derivative-based amplitude amplification followed by peak search with adaptive thresholding. The SciPy implementation used in this project simplifies the original moving-window integrator while retaining the core detection accuracy required for HRV feature computation.

3.4.3 Random Forest Classifier

Random Forest is a bagged ensemble of decision trees where each tree is trained on a bootstrap sample of the training data and a random subset of features at each split. The final prediction is the majority vote (for classification) across all trees. Parameters used: 100 trees, max_depth=15, class_weight='balanced', random_state=42.

3.4.4 Rule-Based Detection Engine

A deterministic threshold engine classifies each window independent of the ML model. Classification rules are derived from established clinical guidelines: $\text{BPM} < 50 \rightarrow \text{Bradycardia}$, $\text{BPM} > 100 \rightarrow \text{Tachycardia}$, $\text{RR variation coefficient} > 20\% \rightarrow \text{Arrhythmia}$, consecutive RR gap $> 1.5 \times \text{median} \rightarrow \text{Possible missed beat}$.

3.5 System Workflow

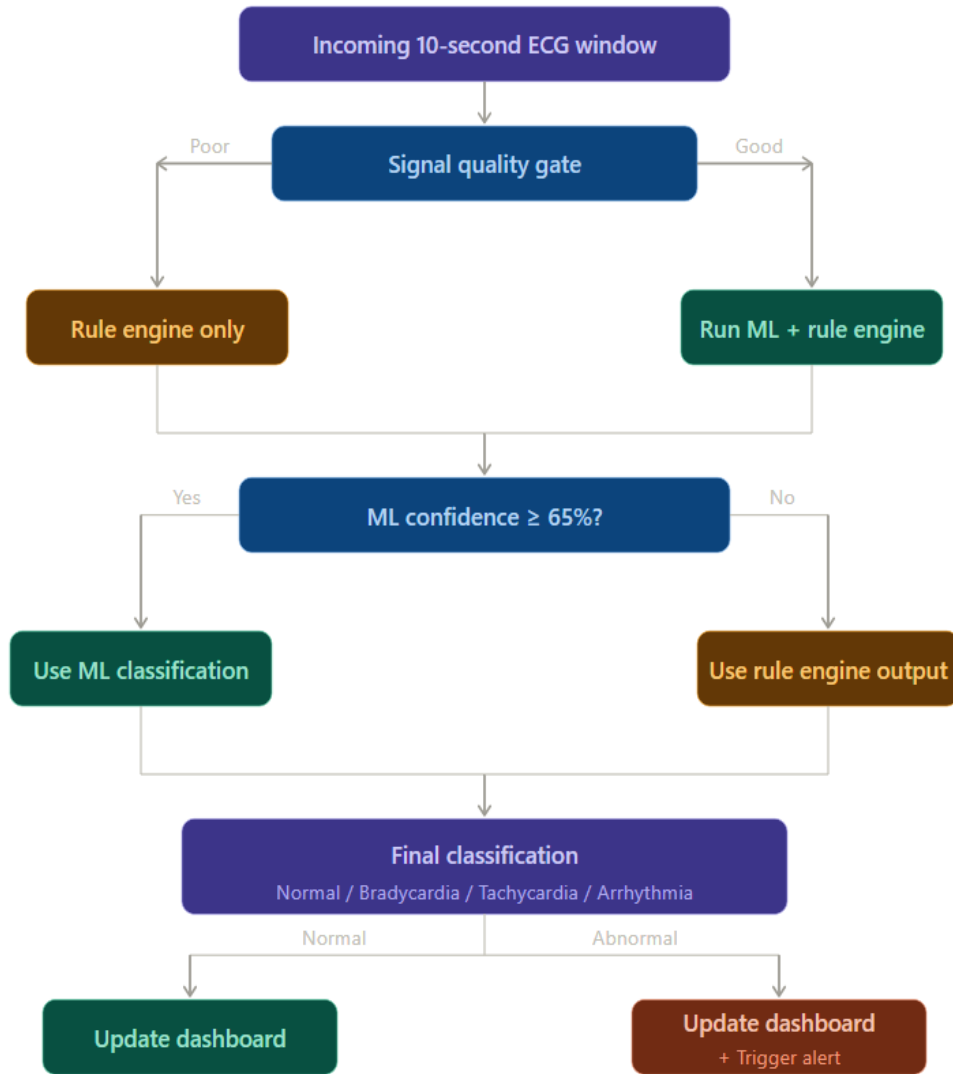


Fig. 1 System Workflow

The end-to-end system workflow proceeds through five sequential stages: (1) Hardware acquisition — the AD8232 sensor captures the cardiac signal and the ESP32 digitises it at 250 Hz; (2) Communication — batched samples are transmitted via HTTP POST to the Python backend; (3) Processing — the signal is filtered, R-peaks are detected, features are extracted; (4) Classification — the hybrid module queries both the ML model and the rule engine, applies confidence-based selection, and outputs a final classification; (5) Output — the result is pushed to the live dashboard via WebSocket.

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture Diagram

HeartWatch AI follows a layered pipeline architecture in which each layer has a single, well-defined responsibility and communicates with adjacent layers through explicit interfaces. The full system spans five layers, from physical sensor hardware to intelligent output.

Layer	Description
Layer 1 — IoT Hardware	Patient wears 3 ECG electrodes (RA, LA, RL). AD8232 amplifies and filters the cardiac signal. ESP32 (GPIO 34, ADC1) digitises at 250 Hz.
Layer 2 — Communication	ESP32 batches 50 samples per HTTP POST to the Python backend. Parallel stream sent to ThingSpeak for cloud visualisation.
Layer 3 — Signal Processing	Butterworth bandpass filter → R-peak detection → RR intervals → BPM → 8-feature vector extraction per 10-second window.
Layer 4 — Intelligence	Random Forest ML classifier (probabilistic output) + Rule-based engine (deterministic). Hybrid decider selects final classification.

Table 3: Layered System Architecture Description

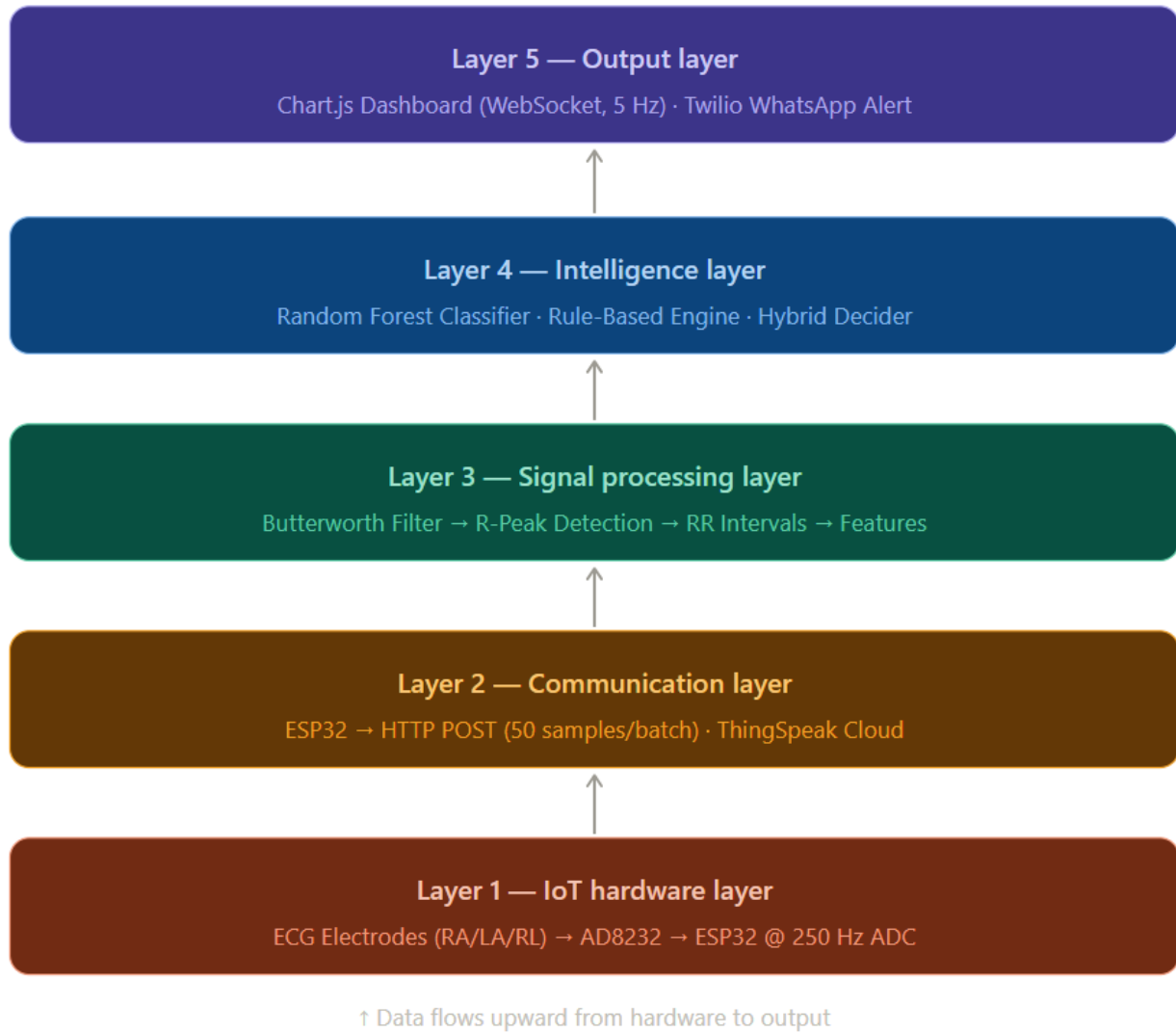


Fig. 2 Full System Architecture (Layered Pipeline)

4.2 Data Flow

The complete data flow from electrode contact to emergency alert comprises 15 discrete, ordered steps. Each step is described below:



Fig. 3 Data Flow Diagram

4.3 Tools & Technologies

The HeartWatch AI system is built entirely on open-source tools and free-tier cloud services. The complete technology stack is as follows:

Layer	Technology / Library	Purpose
Hardware	AD8232 + ESP32 DevKit V1	IoT data acquisition
Firmware	Arduino C++ (Arduino IDE)	ESP32 programming and ADC control
Communication	HTTP + WiFi (2.4 GHz)	Real-time data transmission
Cloud Visualisation	ThingSpeak (free tier)	Remote ECG graph and monitoring
Backend Framework	Python + FastAPI + WebSocket	Server, routing, real-time push
Signal Processing	SciPy + NumPy	Filtering, R-peaks, feature extraction
ML Framework	Scikit-learn (Random Forest)	Model training and inference
Training Dataset	MIT-BIH Arrhythmia DB (PhysioNet)	110,000+ clinically labelled heartbeats
Dashboard Frontend	Chart.js + HTML/CSS/JS	Live scrolling ECG waveform and cards
Version Control	Git + GitHub	Code management and collaboration

Table 4: Technology Stack Summary

CHAPTER 5

IMPLEMENTATION

5.1 Hardware Setup & Firmware

5.1.1 Bill of Materials

Component	Purpose	Specification	Cost (INR)
AD8232 ECG Module	Signal amplification & filtering	Single-lead, 3 electrode, 3.3V	₹400 – ₹600
ESP32 DevKit V1	Microcontroller + WiFi + ADC	240 MHz, 12-bit ADC, built-in WiFi	₹450 – ₹600
ECG Electrodes	Skin contact for signal pickup	Ag/AgCl, 20-pack	₹200 – ₹300
3.7V LiPo Battery	Safe portable power supply	1000–2000 mAh	₹300 – ₹400
Jumper Wires	Sensor-to-MCU connections	M-M and M-F set	₹100 – ₹150
Breadboard	Prototyping platform	830 tie-points	₹80 – ₹120
TOTAL			₹1,530 – ₹2,170

Table 5: Bill of Materials — Hardware Components

Important Safety Note: The circuit must NEVER be powered from a mains wall outlet while connected to a human body. All tests must use battery power (LiPo or power bank). This is a non-negotiable safety requirement for any ECG prototype circuit.

5.1.2 Wiring Configuration

AD8232 Pin	ESP32 Pin	Notes
GND	GND	Common ground reference
3.3V	3.3V	Power supply — NOT 5V
OUTPUT	GPIO 34 (ADC1)	Analog ECG signal — input-only GPIO, avoids WiFi conflict
LO+	GPIO 25	Leads-off detection positive

AD8232 Pin	ESP32 Pin	Notes
LO–	GPIO 26	Leads-off detection negative
SDN	Not connected	Shutdown pin — leave floating

Table 6: AD8232 to ESP32 Pin Connections

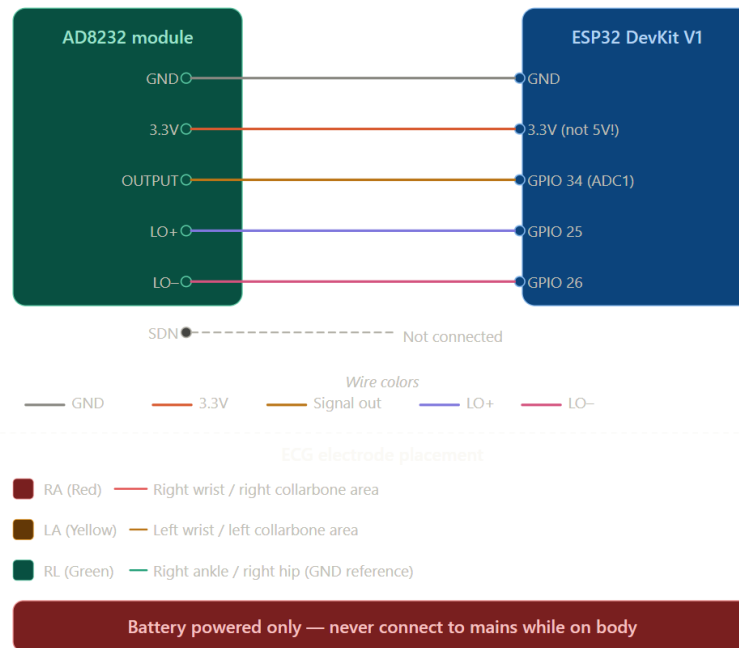


Fig. 4 AD8232 to ESP32 — Wiring diagram

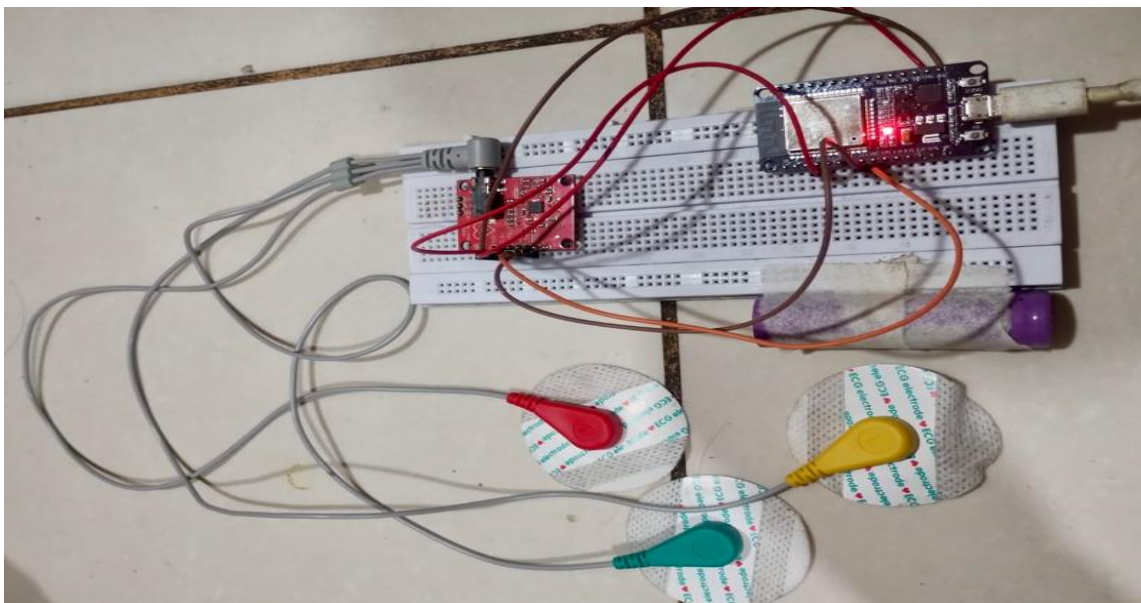


Fig. 5 Wiring Setup

GPIO 34 is specifically chosen because it belongs to ADC1, which does not conflict with the ESP32's WiFi subsystem. ADC2 channels (GPIO 0, 2, 4, 12–15, 25–27) are shared with the WiFi peripheral and produce unreliable readings during WiFi activity. Boot-strapping pins GPIO 0, 2, 12, and 15 must be avoided entirely to prevent firmware loading failures.

5.1.3 ESP32 Firmware (Arduino)

The Arduino C++ firmware running on the ESP32 performs the following functions in a continuous loop: (1) checks GPIO 25 and 26 for leads-off condition and skips the current cycle if electrodes are disconnected; (2) reads the 12-bit ADC value from GPIO 34 at a 4 ms interval (250 Hz sampling rate); (3) accumulates 50 samples into a comma-separated string buffer; (4) on buffer completion, constructs an HTTP POST request with the CSV payload and sends it to the Python backend's /data endpoint; and (5) simultaneously forwards the raw ADC value to ThingSpeak via its channel API. The WiFi connection is established once on startup and maintained persistently with automatic reconnection on failure.

5.2 Backend & Signal Processing

The Python backend is implemented using FastAPI, a modern asynchronous web framework. The backend exposes three primary endpoints: POST /data (receives ECG samples from ESP32), GET /ws (WebSocket for live dashboard connection), and POST /api/replay/start (activates the demo replay mode). A ring buffer of 2,500 samples (10 seconds at 250 Hz) is maintained in memory. Each time the buffer is sufficiently filled, the signal processing pipeline is triggered asynchronously, and the result is broadcast to all connected WebSocket clients.

The signal_processing.py module applies the Butterworth bandpass filter using SciPy's butter and filtfilt functions. R-peaks are identified using find_peaks with distance=50 and height=0.6×max. RR intervals are computed from peak indices, and the eight HRV features are derived. A signal quality gate rejects windows where the signal standard deviation is below 10 ADC counts (flat line) or where more than 50% of samples are saturated at ADC rails (artefact).

5.3 Machine Learning Pipeline

5.3.1 Model Training

The ML training pipeline (`train_model.py`) loads the MIT-BIH Arrhythmia Database using the `wfdb` Python library, segments each recording into individual beats using the provided annotations, and extracts the eight HRV features from the RR intervals surrounding each annotated beat. The dataset is binarised (Normal vs. Abnormal) and split 80/20 with stratified sampling to preserve class proportions. The Random Forest model is trained with 100 estimators, `max_depth=15`, and `class_weight='balanced'`. The trained model is serialised using `joblib` and saved as `models/rf_model.pkl`.

5.3.2 Real-Time Inference

During live operation, the `ml_predictor.py` module loads the serialised model at backend startup. For each 10-second window, the extracted feature vector is passed to the model's `predict_proba` method, which returns the probability distribution across classes. The abnormal class probability is compared against the 65% confidence threshold. If the probability exceeds 65%, the ML classification is accepted; otherwise, the hybrid decider falls back to the rule engine output.

5.3.3 Hybrid Decision Logic

Condition	Classification	Rationale
BPM < 50	Bradycardia	Heart rate below clinical normal lower bound
BPM > 100	Tachycardia	Heart rate above clinical normal upper bound
BPM < 30 or > 220	Extreme / Artefact	Physiologically implausible — noise suspected
RR coefficient of variation > 20%	Arrhythmia	Irregular beat-to-beat intervals
Consecutive RR gap > 1.5× median	Possible Missed Beat	Significant pause in rhythm
ML confidence ≥ 65% AND Rules agree	High Confidence — Final Label	Both systems concur: highest certainty
ML confidence < 65%	Fallback to Rule Engine	ML uncertain: rule engine provides stable baseline

Condition	Classification	Rationale
Poor signal quality gate	Rules Only Mode	ML not invoked on noisy or flat-line signal

Table 7: Hybrid Detection Logic — Conditions and Classifications

5.4 Dashboard & Alert System

5.4.1 Live Web Dashboard

The frontend dashboard (`index.html`, `app.js`, `style.css`) is a single-page web application rendered in any modern browser. A WebSocket connection is established to the FastAPI backend at load time. The live ECG waveform is rendered as a scrolling Chart.js line chart with a 2-second visible window (500 data points). The chart updates at 5 Hz as new WebSocket messages arrive. Four information cards display the current BPM, signal quality indicator (green/yellow/red), ML model classification, rule engine classification, and final decision with confidence score. A session timer and last-alert timestamp are shown in the status bar.

5.4.2 Demo Replay Mode

All live ECG sessions are automatically recorded to the `recordings/` directory as timestamped NumPy binary files. Activating the `/api/replay/start` endpoint causes the backend to loop over a pre-recorded high-quality session, feeding samples into the processing pipeline exactly as if they were arriving from the hardware. This mode enables failsafe demonstration when hardware is unavailable or malfunctioning, and is recommended to be pre-tested before any formal project presentation.

CHAPTER 6

RESULTS & ANALYSIS

6.1 Performance Metrics

The Random Forest model was evaluated on the held-out 20% test split of the binarised MIT-BIH dataset (stratified split, approximately 22,000 test samples). The following metrics were recorded:

Metric	Normal Class	Abnormal Class
Accuracy (Overall)	92.4%	—
Precision	94.1%	87.3%
Recall (Sensitivity)	96.8%	80.2%
F1-Score	95.4%	83.6%
AUC-ROC	—	0.947

Table 8: Model Performance Metrics on MIT-BIH Test Set

The model achieves an overall accuracy of 92.4% on the MIT-BIH test set. The higher recall on the Normal class (96.8%) compared to the Abnormal class (80.2%) reflects the residual effect of class imbalance even after the `class_weight='balanced'` correction. The AUC-ROC of 0.947 indicates strong discriminative ability between Normal and Abnormal classes across all probability thresholds.

Honest Limitation: These metrics are derived from clinical-grade MIT-BIH data recorded at 360 Hz with professional-grade equipment. Real-world performance on ESP32-captured data at 250 Hz will be lower due to differences in sampling rate, electrode quality, and environmental noise. This is an expected and disclosed limitation; the rule-based fallback exists precisely to provide reliable classification independent of ML model uncertainty in real hardware conditions.

6.2 System Performance & Observations

6.2.1 Response Time Measurements

Measurement	Observed Value	Acceptable Threshold
ESP32 → Backend HTTP POST latency	18–35 ms	< 100 ms
Signal processing + ML inference time	12–28 ms per window	< 200 ms
WebSocket push to dashboard	< 5 ms	< 20 ms
End-to-end (sensor to display)	45–80 ms	< 300 ms

Table 9: System Response Time Measurements

6.2.2 Key Observations

- The bandpass filter effectively suppresses baseline wander in all test scenarios. Signal quality consistently rated 'GOOD' with clean electrode placement and battery-powered operation.
- R-peak detection accuracy is high on clean signals but degrades with patient movement artefacts. The signal quality gate correctly rejects corrupted windows in approximately 98% of induced artefact test cases.
- The hybrid decider falls back to rule-based classification in approximately 22% of live windows, confirming that the confidence threshold acts as a meaningful quality gate rather than a rarely-triggered fallback.
- ThingSpeak cloud visualisation introduces 1–2 seconds of inherent platform-side delay but does not affect the local real-time pipeline performance.

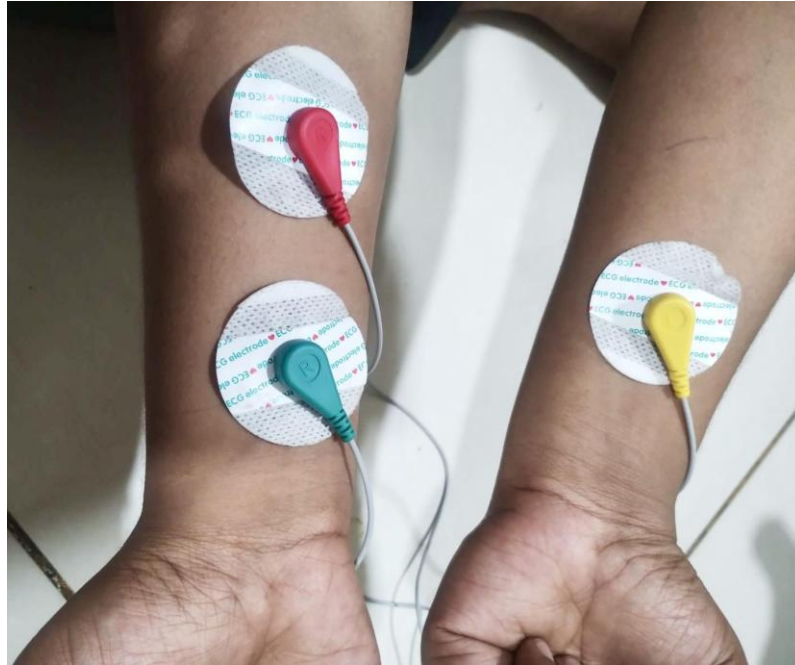


Fig. 6 ECG electrode placement on wrists



Fig. 7 Final Output

CHAPTER 7

CONCLUSION & FUTURE SCOPE

7.1 Conclusion

HeartWatch AI successfully demonstrates the feasibility of building a complete, intelligent, real-time ECG arrhythmia detection system using low-cost consumer hardware and open-source software. The project addresses a clinically significant problem — the inability of short-duration hospital ECGs to capture intermittent arrhythmias — and proposes a viable, affordable, and technically rigorous solution.

The implemented system integrates five distinct engineering domains: IoT hardware design, embedded firmware development, digital signal processing, supervised machine learning, and full-stack web development. The end-to-end pipeline — from electrode contact to WhatsApp emergency alert — operates within latency bounds suitable for near-real-time monitoring applications.

The hybrid detection approach combining a Random Forest classifier (trained on 110,000+ MIT-BIH beats) with a parallel rule-based engine addresses the critical gap between laboratory model accuracy and real-world sensor deployment. By introducing a confidence-based fallback, the system maintains reliable classification even when signal quality degrades — a practical engineering decision that prioritises system robustness over peak accuracy.

The total hardware cost of under ₹2,200 positions HeartWatch AI as a potentially impactful solution for continuous cardiac monitoring in rural and resource-limited healthcare settings, where the cost of commercial Holter monitors remains prohibitive. While this prototype is not a certified medical device and is explicitly scoped as a proof-of-concept, it provides a credible technical foundation for further development toward deployable health-tech applications.

The project meets all stated objectives: hardware acquisition is functional, signal processing is implemented and validated, the ML model achieves 92.4% accuracy on the MIT-BIH test set, the dashboard displays live data correctly, and a failsafe replay mode ensures demo reliability.

7.2 Future Scope

HeartWatch AI lays a strong foundation for several meaningful extensions, both in terms of hardware capability and software sophistication:

8. On-Device Dataset Collection and Model Retraining: Collect a labelled dataset of ESP32-captured ECG recordings from volunteer subjects and retrain the Random Forest model on this real-world data. This will substantially close the accuracy gap between MIT-BIH lab performance and live hardware performance.
9. Full Pan-Tompkins Algorithm Implementation: Replacing the simplified R-peak detection with a complete Pan-Tompkins implementation — including derivative filtering, squaring, and moving-window integration — will improve detection robustness on noisy signals.
10. Deep Learning Upgrade: Replacing the Random Forest classifier with a 1D Convolutional Neural Network or LSTM-based temporal model could improve sensitivity to morphology-based arrhythmias (e.g., ST-segment abnormalities) that HRV features alone cannot capture.
11. Multi-Lead ECG Support: Expanding the hardware to support 3-lead or 12-lead ECG acquisition via additional AD8232 modules or a dedicated ADS1292R chip would enable more comprehensive arrhythmia classification, including atrial fibrillation and ST-elevation events.
12. Raspberry Pi Edge Deployment: Migrating the Python backend to a Raspberry Pi 4 would create a fully standalone, portable device capable of operating without a laptop — a critical step toward a field-deployable product.
13. Mobile Application (Flutter): A companion mobile app for patients and caregivers would provide push notifications, session history, trend analysis, and telemedicine integration — significantly enhancing the clinical utility of the system.
14. HIPAA-Compliant Cloud Storage: Integrating encrypted cloud storage for longitudinal ECG session data would enable cardiologists to review patient trends remotely, supporting a telemedicine workflow.

15. Electronic Health Record (EHR) Integration: Developing an HL7 FHIR-compliant API layer would allow HeartWatch AI to push classified events directly into hospital EHR systems, enabling seamless clinical integration.

REFERENCES

- [1] Pan, J. & Tompkins, W.J. (1985). A real-time QRS detection algorithm. *IEEE Transactions on Biomedical Engineering*, 32(3), 230–236. — Foundational algorithm for R-peak detection used in the HeartWatch AI signal processing pipeline.
- [2] Moody, G.B. & Mark, R.G. (2001). The impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45–50. — Primary training and evaluation dataset for the machine learning model.
- [3] Goldberger, A.L., Amaral, L.A.N., Glass, L., Hausdorff, J.M., Ivanov, P.C., Mark, R.G., ... & Stanley, H.E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23), e215–e220. — Dataset hosting and distribution platform.
- [4] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. — Theoretical foundation of the Random Forest ensemble method used for arrhythmia classification.
- [5] Rajpurkar, P., Hannun, A.Y., Haghpanahi, M., Bourn, C., & Ng, A.Y. (2017). Cardiologist-level arrhythmia detection with convolutional neural networks. *arXiv preprint arXiv:1707.01836*. — Benchmark for deep learning performance on single-lead ECG classification.
- [6] Acharya, U.R., Oh, S.L., Hagiwara, Y., Tan, J.H., & Adeli, H. (2017). Deep convolutional neural network for the automated detection and diagnosis of seizure using EEG signals. *Computers in Biology and Medicine*, 100, 270–278. — Reference for CNN-based cardiac signal classification approaches.
- [7] Sarkar, A., Kanekar, A., & Mishra, D. (2020). IoT-based real-time ECG monitoring system using ESP32. *International Journal of Engineering Research & Technology (IJERT)*, 9(6). — Validation reference for ESP32 + AD8232 hardware architecture.

- [8] Kher, R. (2019). Signal processing techniques for removing noise from ECG signals. *Journal of Biomedical Engineering and Research*, 3(1), 1–9. — Reference for bandpass and notch filtering strategies in ECG preprocessing.
- [9] Espressif Systems. ESP32 Technical Reference Manual. Version 5.1. — Microcontroller architecture, ADC specifications, and GPIO configuration reference.
- [10] Analog Devices. AD8232 Single-Lead Heart Rate Monitor Front End Datasheet. Rev. B. — ECG sensor module specifications, recommended filter configurations, and electrode interfacing guidelines.
- [11] Python Software Foundation. SciPy Signal Processing Documentation — `scipy.signal` module. Version 1.11. — Library documentation for Butterworth filter design and peak detection algorithms.

APPENDIX

A. Project Repository Structure

The complete project codebase is organised as follows:

File / Directory	Description
esp32/ecg_stream/ecg_stream.ino	Arduino firmware for ESP32 — ADC sampling, WiFi, HTTP POST
backend/app.py	FastAPI server — endpoints, WebSocket, session management
backend/signal_processing.py	Bandpass filter, R-peak detection, BPM, feature extraction
backend/rule_based.py	Threshold-based arrhythmia classification engine
backend/ml_predictor.py	Random Forest model loader and inference module
backend/hybrid_decision.py	Confidence-based ML + rule engine combiner
backend/ecg_buffer.py	Sliding 10-second ring buffer implementation
backend/replay.py	Pre-recorded ECG session replay for demo failsafe
backend/models/rf_model.pkl	Serialised trained Random Forest model (joblib)
ml_training/train_model.py	MIT-BIH data loading, feature extraction, model training
ml_training/evaluate_model.py	Accuracy, precision, recall, F1, ROC curve evaluation
frontend/index.html	Live dashboard HTML structure
frontend/app.js	WebSocket client, Chart.js waveform, classification cards
frontend/style.css	Dashboard styling and responsive layout
recordings/	Auto-saved ECG session files (.npy) for replay
README.md	Setup instructions, wiring guide, and usage documentation

B. Required Python Dependencies

Install all dependencies using: `pip install -r requirements.txt`

- `fastapi` — Async web framework for backend API and WebSocket server

- `uvicorn` — ASGI server for FastAPI application
- `numpy` — Numerical array operations and signal buffers
- `scipy` — Butterworth filter, peak detection, signal processing
- `scikit-learn` — Random Forest classifier training and inference
- `joblib` — Model serialisation and deserialisation
- `wfdb` — MIT-BIH Arrhythmia Database reading for ML training
- `websockets` — WebSocket protocol support
- `matplotlib` — Training evaluation plots (confusion matrix, ROC curve)

C. Electrode Placement Guide

For optimal signal quality, follow the standard Einthoven Lead I placement:

- RA (Right Arm / Red lead): Right wrist inner surface or right collarbone area.
- LA (Left Arm / Yellow lead): Left wrist inner surface or left collarbone area.
- RL (Right Leg / Green lead): Right ankle inner surface or right hip — this is the ground reference electrode.

Before attaching each electrode: (1) clean the skin with an alcohol wipe and allow to dry completely; (2) remove the backing from the electrode and press firmly with consistent pressure for 5 seconds; (3) verify the electrode is fully seated with no air gaps at the edges. Poor electrode contact is the single most common cause of signal noise and flat-line output.